

Practicing Window Function

Saturday, August 30, 2025 8:02 PM

Create table Script

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(50),  
    salary INT,  
    joining_date DATE,  
    sales_amount INT  
);
```

Insert Records Script

```
INSERT INTO employees (emp_id, emp_name, department, salary, joining_date, sales_amount) VALUES  
(1, 'Alice', 'HR', 60000, '2020-01-15', 200),  
(2, 'Bob', 'HR', 55000, '2021-03-10', 150),  
(3, 'Charlie', 'HR', 70000, '2019-07-23', 400),  
(4, 'David', 'IT', 80000, '2018-06-11', 600),  
(5, 'Eva', 'IT', 75000, '2021-08-01', 500),  
(6, 'Frank', 'IT', 82000, '2019-09-19', 650),  
(7, 'Grace', 'Finance', 90000, '2020-12-30', 700),  
(8, 'Hannah', 'Finance', 87000, '2022-01-25', 550),  
(9, 'Ian', 'Finance', 91000, '2018-02-14', 750),  
(10, 'Jack', 'Sales', 65000, '2021-05-05', 300),  
(11, 'Karen', 'Sales', 67000, '2020-09-15', 450),  
(12, 'Leo', 'Sales', 70000, '2019-11-20', 500);
```

Windows and Ranks

--Row numbering based on salary

```
select  
emp_name, row_number() over(order by salary desc)  
from Employees;
```

Output

emp_name	row_number() over(order by salary desc)
Ian	1
Grace	2
Hannah	3
Frank	4
David	5
Eva	6

--Row numbering per department , Assign a row number to each employee within their department, ordered by salary (highest first).

```
select emp_name, row_number() over(partition by department order by salary desc) from Employees;
```

Output

emp_name	row_number() over(partition by department order by salary desc)
Ian	1
Grace	2
Hannah	3
Charlie	1
Alice	2
Bob	3

Ranking employees by salary (overall)

Show each employee's salary rank across the whole company using RANK().

```
select emp_name, rank() over (order by salary) from Employees;
```

Output

emp_name	rank() over (order by salary)
Bob	1
Alice	2
Jack	3
Karen	4
Charlie	5
Leo	5

Dense ranking per department

Rank employees within each department by sales, but without gaps (DENSE_RANK).

```
select
  emp_name,
  department,
  dense_rank() over(partition by department order by sales_amount desc) as sales_rank
from employees;
```

emp_name	department	sales_rank
Ian	Finance	1
Grace	Finance	2
Hannah	Finance	3
Charlie	HR	1
Alice	HR	2
Bob	HR	3

without gaps means DENSE_RANK() continues with the next integer, instead of skipping numbers like RANK() does.

Find the highest salary per department

Use ROW_NUMBER() to return only the top 1 employee per department by salary.

- The LIMIT clause cannot be used **inside** the OVER() clause.
- To return only the top employee per department, you need to **use a subquery or a Common Table Expression (CTE)** to first assign row numbers, then **filter** to only keep the rows with ROW_NUMBER = 1.

Top 1 salary from each department

```
with rankingTable as (
  select emp_name, department, salary,
  row_number() over(partition by department order by salary desc) as rn
  from employees
)
select emp_name, department, salary from rankingTable
where rn=1;
```

Output

emp_name	department	salary
Ian	Finance	91000
Charlie	HR	70000
Frank	IT	82000
Leo	Sales	70000

Cumulative salary (running total)

For each department, calculate a running total of salaries ordered by joining_date.

```
select department, sum(salary) over(partition by department order by joining_date desc) as running_sum from employees
```

department	running_sum
Finance	87000
Finance	177000
Finance	268000
HR	55000
HR	115000
HR	185000

Intermediate comparison and averages

Previous & next salary

Show each employee's salary along with the previous and next employee's salary in their department (LAG, LEAD).

```
select emp_name, department,
lag(salary,1) over(partition by department order by salary desc ) as prev_salary,
salary,
lead(salary,1) over(partition by department order by salary desc) as next_salary
from employees;
```

Output

emp_name	department	prev_salary	salary	next_salary
Ian	Finance		91000	90000
Grace	Finance	91000	90000	87000
Hannah	Finance	90000	87000	
Charlie	HR		70000	60000
Alice	HR	70000	60000	55000
Bob	HR	60000	55000	

Salary difference from previous employee

For each employee, find the difference between their salary and the previous one in their department.

```
select emp_name, department,
lag(salary,1) over(partition by department order by salary desc ) -
lead(salary,1) over(partition by department order by salary desc) as salary_diff
from employees;
```

Output

emp_name	department	salary_diff
Ian	Finance	
Grace	Finance	4000
Hannah	Finance	
Charlie	HR	
Alice	HR	15000
Bob	HR	

Department salary average vs employee salary

For each employee, compare their salary with their department's average salary.

```
select emp_name, salary ,
avg(salary) over(partition by department) as average_dept_salary
from employees
```

Output

emp_name	salary	average_dept_salary
Grace	90000	89333.33333333333
Hannah	87000	89333.33333333333
Ian	91000	89333.33333333333
Alice	60000	61666.666666666664
Bob	55000	61666.666666666664
Charlie	70000	61666.666666666664

Find employees above department average

List employees whose salary is above the average salary in their department.

Note:- You **can't reference a window function alias (average_dept_salary)** in the WHERE clause — window functions are computed **after** WHERE filtering.

Solution subquery

```
SELECT *
FROM (
  SELECT
    emp_name,
    department,
    salary,
    AVG(salary) OVER (PARTITION BY department) AS average_dept_salary
  FROM employees
```

```
) AS sub
WHERE salary > average_dept_salary;
```

Output

emp_name	department	salary	average_dept_salary
Grace	Finance	90000	89333.3333333333
Ian	Finance	91000	89333.3333333333
Charlie	HR	70000	61666.6666666666
David	IT	80000	79000
Frank	IT	82000	79000
Leo	Sales	70000	67333.3333333333

Moving average salary

Calculate the moving average of salary with a window of 2 previous + current row.

```
select emp_name,
avg(salary) over(ORDER BY salary ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) as average_salary
from employees
```

Output

emp_name	average_salary
Bob	55000
Alice	57500
Jack	60000
Karen	64000
Charlie	67333.3333333333
Leo	69000

Level 3 :- Advanced business problems

Top 3 employees per department by salary

Return only the top 3 paid employees from each department.

For a same-problem scenario, a Common Table Expression (CTE) is generally the better choice because it allows you to define a temporary result set once and reference it multiple times within a single query, improving readability, reusability, and maintainability.

```
WITH department_wise_salary AS (
SELECT
    emp_name,
    department,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees
)
SELECT
    emp_name,
    department,
    salary
FROM department_wise_salary
WHERE dept_rank <= 3;
```

Output

emp_name	department	salary
Ian	Finance	91000
Grace	Finance	90000
Hannah	Finance	87000
Charlie	HR	70000
Alice	HR	60000
Bob	HR	55000

Bottom 2 employees per department by sales

Find the lowest 2 sales performers in each department.

```
WITH ranked_sales AS (
SELECT
    emp_name,
    department,
    sales_amount,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY sales_amount ASC) AS row_rank
```

```

FROM employees
)
SELECT *
FROM ranked_sales
WHERE row_rank <= 2;

```

emp_name	department	sales_amount	row_rank
Hannah	Finance	550	1
Grace	Finance	700	2
Bob	HR	150	1
Alice	HR	200	2
Eva	IT	500	1
David	IT	600	2

Percentile salary rank (NTILE)

Divide employees into 4 salary quartiles across the company.

```

SELECT
    emp_name,
    salary,
    NTILE(4) OVER (ORDER BY salary) AS salary_quartile
FROM employees;

```

Output

emp_name	salary	salary_quartile
Bob	55000	1
Alice	60000	1
Jack	65000	1
Karen	67000	2
Charlie	70000	2
Leo	70000	2

First & last joining employee per department

Use FIRST_VALUE and LAST_VALUE to get the earliest and latest hire in each department.

```

SELECT DISTINCT
    department,
    FIRST_VALUE(emp_name) OVER (PARTITION BY department ORDER BY joining_date) AS first_joiner,
    LAST_VALUE(emp_name) OVER (
        PARTITION BY department
        ORDER BY joining_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_joiner
FROM employees;

```

Output

department	first_joiner	last_joiner
Finance	Ian	Hannah
HR	Charlie	Bob
IT	David	Eva
Sales	Leo	Jack

Salary gap analysis

For each employee, calculate how much less they earn than the highest-paid person in their department.

```

WITH maxSalary AS (
    SELECT
        emp_name,
        department,
        salary,
        MAX(salary) OVER (PARTITION BY department) AS max_department_salary
    FROM employees
)
SELECT
    emp_name,
    department,
    salary,
    max_department_salary,
    max_department_salary - salary AS salary_gap
FROM maxSalary;

```

Output

emp_name	department	salary	max_department_salary	salary_gap
Grace	Finance	90000	91000	1000
Hannah	Finance	87000	91000	4000

Ian	Finance	91000	91000	0
Alice	HR	60000	70000	10000
Bob	HR	55000	70000	15000
Charlie	HR	70000	70000	0

Expert complex Analysis

Find salary growth trend

For each department, show each employee's salary and whether it increased compared to the previous joining employee (CASE WHEN salary > LAG(salary))

```
WITH joining AS (
  SELECT
    emp_name,
    department,
    joining_date,
    salary,
    LAG(salary, 1) OVER (PARTITION BY department ORDER BY joining_date) AS prev_salary,
    CASE
      WHEN salary > LAG(salary, 1) OVER (PARTITION BY department ORDER BY joining_date) THEN 'Increased'
      WHEN salary < LAG(salary, 1) OVER (PARTITION BY department ORDER BY joining_date) THEN 'Decreased'
      ELSE 'First Employee'
    END AS salary_trend
  FROM employees
)
SELECT *
FROM joining;
```

Output

emp_name	department	joining_date	salary	prev_salary	salary_trend
Ian	Finance	2018-02-14	91000		First Employee
Grace	Finance	2020-12-30	90000	91000	Decreased
Hannah	Finance	2022-01-25	87000	90000	Decreased
Charlie	HR	2019-07-23	70000		First Employee
Alice	HR	2020-01-15	60000	70000	Decreased
Bob	HR	2021-03-10	55000	60000	Decreased

Cumulative sales until each employee

For each department, show cumulative sales_amount ordered by joining date.

```
with csales as
(
  select
    department,
    emp_name,
    joining_date,
    sum(sales_amount)over(partition by department order by joining_date) as datewise_cSales
  from employees
)
select * from csales
```

Output

department	emp_name	joining_date	datewise_cSales
Finance	Ian	2018-02-14	750
Finance	Grace	2020-12-30	1450
Finance	Hannah	2022-01-25	2000
HR	Charlie	2019-07-23	400
HR	Alice	2020-01-15	600
HR	Bob	2021-03-10	750

Find employees who joined earlier but earn less

Return employees whose salary is less than someone who joined later in the same department.

```
SELECT
  emp_name,
  department,
  joining_date,
  salary,
  CASE
    WHEN salary < LEAD(salary, 1) OVER (PARTITION BY department ORDER BY joining_date) THEN
    'less'
    WHEN salary > LEAD(salary, 1) OVER (PARTITION BY department ORDER BY joining_date) THEN
    'high'
    ELSE 'same'
  END AS salary_trend
FROM employees;
```

Output

emp_name	department	joining_date	salary	salary_trend
Ian	Finance	2018-02-14	91000	high
Grace	Finance	2020-12-30	90000	high
Hannah	Finance	2022-01-25	87000	same
Charlie	HR	2019-07-23	70000	high
Alice	HR	2020-01-15	60000	high
Bob	HR	2021-03-10	55000	same

Running max/min salary

For each employee (ordered by joining_date), compute the maximum and minimum salary so far in their department.

Note:- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW ensures the aggregation works **from the first employee to the current one** within the department.

```
SELECT
    emp_name,
    department,
    salary,
    joining_date,
    MAX(salary) OVER (
        PARTITION BY department
        ORDER BY joining_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS maximum_salary,

    MIN(salary) OVER (
        PARTITION BY department
        ORDER BY joining_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS minimum_salary
FROM employees;
```

Output

emp_name	department	salary	joining_date	maximum_salary	minimum_salar
Ian	Finance	91000	2018-02-14	91000	91000
Grace	Finance	90000	2020-12-30	91000	90000
Hannah	Finance	87000	2022-01-25	91000	87000
Charlie	HR	70000	2019-07-23	70000	70000
Alice	HR	60000	2020-01-15	70000	60000
Bob	HR	55000	2021-03-10	70000	55000